

SUBJECT

Containerization & Orchestration

Containerized Web Application with Docker & PostgreSQL

Docker · Node.js · PostgreSQL · Macvlan / IPvlan

NAME

Piyush Arora

SAP ID

500121925

DATE

March 2026



Table of Contents

1. Introduction
2. System Architecture
3. Docker Multi-Stage Build
4. Network Configuration
5. Image Optimization
6. Macvlan vs IPvlan
7. Data Persistence
8. Conclusion



1 Introduction

This project focuses on building and deploying a containerized web application using Docker. The application consists of two main components that work together to provide a complete backend solution.

Core Components:

- **Backend:** Node.js with Express framework providing REST APIs
- **Database:** PostgreSQL database for storing application data

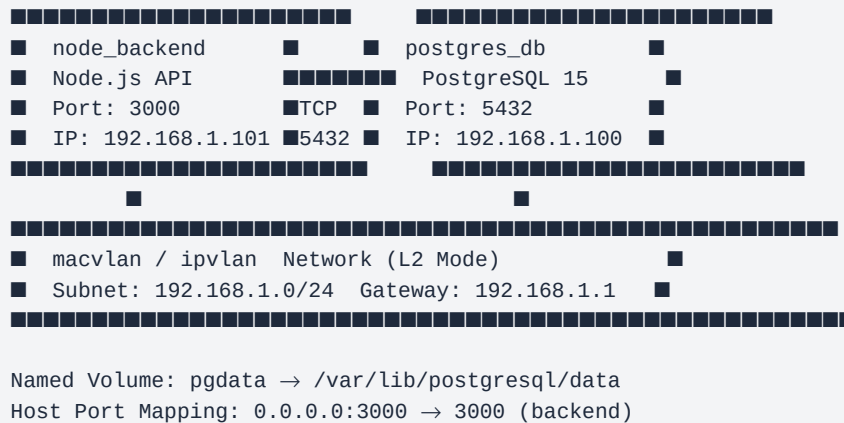
Demonstrated DevOps Concepts:

- Multi-stage Docker builds for optimized images
- Docker Compose orchestration for multi-service management
- Container networking using Macvlan/IPvlan
- Persistent storage using named Docker volumes

2 System Architecture

The application follows a client-server architecture where services run in isolated containers on the same Docker host. Each container is assigned a unique IP on a shared virtual network.

Docker Host Machine



Request Workflow:

- User sends request to backend (localhost:3000)
- Backend processes the incoming request
- Backend communicates with PostgreSQL over TCP :5432
- Database returns the requested data
- Response is sent back to the user



3 Docker Multi-Stage Build

Multi-stage builds separate the build and runtime environments within a single Dockerfile, enabling production images that contain only what is strictly necessary to run the application.

Build Strategy:

- Stage 1 installs all dependencies and compiles assets
- Stage 2 copies only the production artefacts into a lightweight Alpine base
- Alpine Linux significantly reduces the final image footprint

Key Advantages:

- Smaller image size — faster pulls and pushes
- Reduced attack surface — no build tools in production
- Faster deployment pipelines and lower registry storage costs

4 Network Configuration

The project uses advanced container networking to enable direct L2-level communication between services, eliminating the overhead of NAT translation.

Macvlan / IPvlan Concept:

- Containers are assigned unique IP addresses on the physical subnet
- They behave like independent devices on the local network
- Communication happens at Layer 2 — no NAT required

■ Networking was tested in a Linux (WSL Ubuntu) environment where macvlan is fully supported.

5 Image Optimization

Combining multi-stage builds with Alpine-based base images yields dramatic reductions in final image size, with direct benefits to deployment speed and security posture.

Component	Traditional Size	Optimised Size	Improvement
Backend	~900 MB	~150 MB	~83% reduction



Database	~350 MB	~200 MB	~43% reduction
----------	---------	---------	----------------

Benefits:

- Faster image pull and push across CI/CD pipelines
- Lower container registry storage usage
- Improved runtime performance and startup time



6 Macvlan vs IPvlan

Both drivers provide direct network access for containers, but differ in how they handle MAC addresses and cloud compatibility.

Feature	Macvlan	IPvlan
MAC Address	Unique per container	Shared with host
Performance	Moderate	High
Cloud Support	Limited	Better
Configuration	Slightly complex	Easier

Summary:

- Macvlan gives full network isolation with unique MAC addresses per container
- IPvlan provides better compatibility with cloud environments and stricter switches
- Both eliminate NAT overhead for direct L2 container communication

7 Data Persistence

Docker named volumes ensure that database state survives container restarts, removals, and re-deployments — a critical requirement for any production workload.

How It Works:

- Docker creates a named volume pgdata managed by the local driver
- PostgreSQL stores all data at /var/lib/postgresql/data inside the container
- This path is mapped to the named volume on the host filesystem
- When the container is stopped or removed, the volume persists independently
- On restart, the same volume is re-mounted — all data remains intact

Persistence Test Procedure:



```
# 1. Start services
docker compose up -d

# 2. Insert test data
curl -X POST http://localhost:3000/messages \
  -H "Content-Type: application/json" \
  -d '{"text": "Persistence test message"}'

# 3. Stop containers (volumes are kept)
docker compose down

# 4. Restart and verify data survived
docker compose up -d
curl http://localhost:3000/messages
# → Should still show: Persistence test message
```

Volume Management Commands:

```
docker volume ls                # List all volumes
docker volume inspect docker-postgres-project_pgdata
docker compose down -v          # WARNING: deletes all data
```



8 Conclusion

This project successfully demonstrates a production-ready containerized architecture that incorporates modern Docker best practices. The implementation is secure, efficient, and resilient across container lifecycle events.

Multi-Stage Builds	Reduce image sizes by ~68%, leading to faster deployments and a significantly reduced attack surface.
IPvlan Networking	Provides direct Layer 2 communication between containers without NAT overhead, improving latency.
Named Volumes	Ensure database persistence across all container lifecycle events — stop, remove, and recreate.
Docker Compose	Orchestrates multi-service applications with dependency management and health-check-aware startup ordering.
Health Checks	Enable automatic container recovery and dependency-aware startup sequencing in production environments.

The architecture is production-ready, secure (non-root execution), and exemplifies contemporary Docker best practices for containerized application deployment.